The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2], and a computationally demanding one. The most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges, and on the graphs to which they are now routinely applied that quantity is large [7].

Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original. The strongest guarantees are spectral. Spielman and Srivastava [8] show that sampling each edge with probability proportional to its effective resistance, and reweighting the retained edges by the inverse of that probability, yields a subgraph whose Laplacian quadratic form approximates that of the original within a factor $1 \pm \epsilon$ [9], [10]. Spectral approximation is a statement about the Laplacian, and the eigenvectors of the Laplacian encode cuts and connectivity [11], [12]. Community structure is closely related to those quantities without being identical to them. Because effective resistances are themselves expensive to compute, practical variants approximate them. DSpar [13] replaces the resistance of an edge (u, v) by $1/d_u + 1/d_v$, which bounds it within a factor $2/\alpha$ where α is the normalized spectral gap [14], and thereby obtains a spectral sparsifier at the cost of reading the degree sequence. If the properties preserved by sparsification include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. Whether they do is the question this paper addresses. The answer we obtain is conditional. On the networks and detection algorithms examined here, two conditions have to hold together: the algorithm must take the number of communities as an input, and the graph must carry edges whose removal does not destroy its community structure. Where either condition fails we find no gain on the objective, no end-to-end speed benefit, and no gain in recovery that survives comparison with an unsparsified run of a better detector.

The authors are with the Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA. E-mail: {md724, bd344, ikoutis, bader}@njit.edu

Satuluri et al. [15] introduced local sparsification, L-Spar, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. They reported speedups commonly between 10× and 50× with little or no loss of cluster quality, and improved accuracy for two of the four clustering algorithms they tested.

The approach was taken up and extended. Hamann et al. [16] conducted a systematic comparison of structure-preserving sparsification methods for social networks and released reference implementations, which are now distributed in standard graph libraries. Wu and Chen [17] trained a generative model to sparsify graphs specifically for community detection, and report that clustering accuracy is retained while as little as five percent of the edges are kept. Sparsification also entered production: the SimClusters system at Twitter [18] prunes its interaction graph before community detection at industrial scale. Most recently, Chen et al. [19] benchmarked twelve sparsification algorithms against sixteen graph properties on fourteen networks, concluding that no single sparsifier preserves all properties and that the choice should be matched to the downstream task.

The conditions under which the original result was obtained are specific, and none of them travelled with it. The accuracy gain was reported for fixed-k cut partitioners, judged against external ground-truth communities, on dense graphs, and against clustering runs that took hours. The pipeline is now applied to modularity optimizers that choose their own granularity, judged partly by objectives computed on the graph itself, on sparse graphs, and against detection algorithms that run in near-linear time. The same paper reports a synthetic sweep in which the method begins to outperform clustering on the unsparsified graph at an average degree of approximately 50, and to our knowledge no subsequent work examines that threshold.

These extensions also change what has to be measured. The question a sparsify-then-detect pipeline is meant to answer is whether the communities of the original graph can still be recovered after most of its edges are removed. The object of interest is therefore the original graph, and the quantity to be compared is the quality, on that graph, of two partitions: the one obtained by running the detection algorithm on the sparsified graph, and the one obtained by running it on the original graph directly. Both partitions cover the same vertex set, so both can be scored on the same object, and the difference between those two scores is the effect of sparsification.

The sparsifier classes and detection algorithms described above each impose further requirements of the same kind, and several have been noted individually in the literature. Similarity-based sparsifiers [15] remove between-community edges preferentially, and those are the edges that modularity and conductance penalize, so the sparsifier and the quality measure act on the same quantity. Degree-based sparsifiers [13] select on endpoint degrees, and a heavy-tailed degree sequence reproduces several of the resulting effects with no community structure present, so a degree-preserving random graph [20] is needed to separate the two. Detection algorithms that choose their own granularity [4], [5] return finer partitions on a sparsified graph: Chen et al. [19] document community counts rising by three orders of magnitude across a pruning sweep, and Hamann et al. [16] warn that normalized mutual information is inflated when sparsification fragments the graph, recommending chance-corrected indices in its place. Finally, sparsification is itself a computation, and Gottesbüren et al. [21] observe that its overhead can counteract the speedup it is intended to produce. Each requirement is a property of a specific method class rather than a general caveat, and each bounds what a comparison involving that class can conclude. Section III states the requirements and the control that establishes each one.

Applying these requirements, we re-examined the question across seven sparsifiers, spanning the degree-based and similarity-based families that have claimed quality gains, the method of Satuluri et al. among them, and a connectivity-preserving class that cannot fragment a graph by construction; four detection algorithms drawn from the two families identified above; seventeen real networks of up to 25 million edges, seven of which carry the full quality sweep and all seventeen the mechanism measurements; and two families of synthetic benchmark, LFR graphs and stochastic block models, in which average degree is swept over an order of magnitude while community structure is held fixed. One arm of the synthetic study uses a fixed-k partitioner, so that the number of communities is identical in the sparsified and unsparsified conditions and granularity is excluded by construction rather than by matching. The study comprises 23 controlled experiments. Predictions and stopping rules were fixed before each experiment was run, and several of our own hypotheses did not survive them.

Of the two conditions, the second has a location. Average degree is the correlate we can vary under control, and measured against it the transition between the two regimes is sharp within a given family of graphs while its location is not universal. On the LFR benchmark graphs [22] we generate it falls near an average degree of 50, which is also the threshold reported in the original synthetic sweep. On the real networks we test it has already occurred by an average degree of 29 and is absent on every network below 11, so it lies between those two densities. On stochastic block models it occurs no later than on LFR. We therefore state a condition rather than a threshold.

What the transition tracks is how many edges can be removed without destroying community structure, rather than density as such. Injecting spurious edges into a graph makes sparsification more effective, which is the signature that quantity predicts, and average degree is its strongest measured correlate. Degree heterogeneity is not a correlate at all. Plain and degree-corrected block models generated at the same average degree and mixing cross at the same place although their degree variability differs by a factor of ten, and across those graphs the gain correlates with average degree at Spearman $+0.65$ and with degree variability at -0.26. The measurement noise present in how a graph was collected remains a correlate we have not separated.

Where either condition fails, the three ways a benefit could appear all close. For detection algorithms that choose their own granularity, on the sparse real networks we test, no sparsifier improves modularity measured on the original graph beyond run-to-run variation, at any retention that preserves quality, and this holds for every such algorithm we tested. There is no speed benefit either. Measured end to end, including the cost of the sparsifier, the pipeline is no faster than clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget, with one exception on one network, where the surviving recovery gain is instead exceeded by an unsparsified run of a different detection algorithm.

Where both conditions hold, the gain is real. On synthetic graphs with planted communities, with the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels. Recovery turns positive at a lower density than the objective does. On the benchmark graphs, degree-based and uniform sparsification at identical retention do not reproduce the effect, which places it in the selection rule rather than in the edge budget; on the one real network above the transition, a degree-based sampler reproduces it as well, while the two methods that preserve connectivity by construction do not. What the gain requires is a rule that scores individual edges, and we do not identify the property of that rule which matters.

A fixed-k partitioner applied to a sparse graph shows no gain on the objective in any configuration we tested, and a free-granularity optimizer applied to a dense one shows none either once granularity is controlled. Neither condition is sufficient on its own. Which sparsifier is used matters less than either condition, provided its selection signal reflects local structure rather than degree alone. The mechanism is consistent with this: what governs a degree-based sparsifier's effect on modularity is where high-degree-product edges sit relative to community boundaries, which we establish by direct intervention.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one. On the synthetic benchmarks, a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured, and the same holds on all but one of the real networks we test. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. The conditions under which sparsification helps, namely a detection algorithm that takes the number of communities as an input and a graph carrying sufficient removable redundancy, established under controls that exclude granularity by construction, together with the negative territory delimited across seven sparsifiers and four detection algorithms on seven real networks. The negative result is not an artifact of the fragmentation that sparsification induces: a connectivity-preserving sparsifier that cannot disconnect a graph leaves no vertex in a small component, and modularity falls by the same margin. We also report that these methods have a hard retention floor on sparse graphs, so that aggressive sparsification is not merely unhelpful there but unreachable.
- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and modularity are all held exactly fixed.
- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

II. Background and Related Work

III. Evaluation Protocol

A sparsify-then-detect pipeline is a claim about a graph that no longer exists in the pipeline's output. That is the source of every difficulty in evaluating it, and each control below follows from it. We state the controls here and apply them without further comment in Section IV, where their consequences are measured.

A. One yardstick for two partitions

Let $\mathcal{A}$ be a detection algorithm, G the graph whose communities are wanted, and G' a sparsified copy of it. The pipeline returns $P_{\text{sparse}} = \mathcal{A}(G')$ and the alternative to it is $P_{\text{base}} = \mathcal{A}(G)$. Both are partitions of the same vertex set, and the question the pipeline exists to answer is which of them better describes G. Both are therefore scored on G:

$$\Delta Q_{\text{transfer}} = Q(P_{\text{sparse}}, G) - Q(P_{\text{base}}, G), \tag{1}$$

$$\Delta Q_{\text{self}} = Q(P_{\text{sparse}}, G') - Q(P_{\text{base}}, G). \tag{2}$$

The second accounting also appears in this literature. It differs from the first in one place, the graph on which the first term is evaluated, and that difference is not a detail of bookkeeping. Equation (1) measures two partitions with one instrument. Equation (2) changes the instrument between the arm under test and the arm it is tested against, and gives the arm under test an instrument built by deleting edges from the other one.

The requirement is not ours. It is stated, and followed, in the paper that introduced the pipeline [15, §4.3].

Why (2) favours the treatment follows from the objective. Modularity scores a partition P of a graph with m edges by

$$Q(P, G) \;=\; \sum_{C \in P} \left[\frac{m_C}{m} - \gamma \left(\frac{D_C}{2m} \right)^2 \right], \tag{3}$$

with m_C the number of edges inside community C, D_C the sum of degrees in C, and γ a resolution parameter equal to one in the standard definition. Removing edges does not by itself move this quantity for a fixed partition: the intra-community fraction and the subtracted term fall together. What changes is the value an optimizer can reach. A graph with fewer edges admits higher modularity, and does so even when it has no community structure at all [23], so a partition optimized on G' can score well on G' while describing G worse than the baseline does. Equation (2) reads that difference as an improvement.

B. Matching granularity

Sparsification fragments a graph, and a detection algorithm that chooses its own number of communities responds by returning more of them. Agreement measures are not neutral to that. Purity is non-decreasing under refinement. Normalized mutual information rises with the number of clusters, which Hamann et al. [16] identified as making it unsuitable for sparsified networks, pointing to the adjusted Rand index instead [24]. Average best-match F_1 over reference communities rises for the same reason: with more clusters, some cluster is more likely to match a given reference community well. A comparison between partitions of different granularity therefore measures granularity along with whatever else it is meant to measure.

The control is to give the baseline the same freedom. We partition the original graph at a matched community count by raising γ in (3) until the counts agree, and compare the sparsified partition against that rather than against the optimizer's default. If a finer partition of the untouched graph recovers the reference communities as well, the sparsifier has contributed nothing that the resolution parameter does not.

Where the counts cannot be brought together we draw no recovery conclusion. The comparability rule is $|\Delta k|/k < 25\%$, and the cells it excludes are reported as excluded.

C. Subtracting chance

A partition that contains no information at all does not score zero against reference communities, and what it does score depends on how many parts it has. Chance-corrected indices remove this by construction, which is what adjustment means, and we use them wherever the reference communities form a partition.

Where they do not, and the reference communities overlap and are numerous, we use average best-match F_1, which has no such correction. For every arm we therefore generate a random partition with the same community size distribution as the partition being scored, run the same metric on it, and report the result beside the measurement. The floor moves with the same quantity the previous control governs, so the two are applied together and never as substitutes.

D. An equal compute budget

Every detection algorithm used here is stochastic, so one run is one sample from a distribution over partitions, and a pipeline that sparsifies before detecting has spent more time than one run of the detector. Comparing a pipeline against a single baseline run credits the pipeline with the difference in effort.

The registered control gives the baseline as many independent restarts as fit the pipeline's wall clock and takes the best of them. This control has a weakness that runs in the treatment's favour: when the pipeline is fast, the budget it buys is small, and a small number of restarts may fail to sample a good mode of a multi-modal algorithm. The baseline then looks worse than the algorithm is, and the pipeline inherits the difference.

We therefore report two baselines. The compute-matched one is the fair accounting of what the pipeline costs. The best of a fixed number of restarts is a strictly more generous baseline, which can only handicap the treatment and never manufacture a gain. A result counts as surviving only if it survives both. Since the controls compose rather than substitute, the granularity control is applied to results that have already passed this one.

E. Nulls

Two quantities that this literature reads as evidence about community structure can be produced without any community structure present. Each needs a null that detects the substitution.

a) *A degree sequence without communities:* A degree-based sparsifier selects on a function of the degree sequence alone, so any effect it produces on a heavy-tailed graph may follow from that sequence and not from the communities in it. The control is a degree-preserving rewiring [20], generated by double edge swaps, which holds every vertex degree exactly while destroying the community structure. A signal that reappears on the rewiring is not evidence about communities, whatever its size on the real graph.

b) *Weights without their placement:* A related family of proposals deletes no edges and instead reweights each one by a similarity score. The corresponding null permutes the computed weights uniformly at random across the edges. This preserves the weight distribution exactly and destroys every association between a weight and the structure that produced it, so a gain that survives the permutation is a gain from weight heterogeneity and not from similarity. We are not aware of this control being used in this literature.

F. Realized retention

Retention is the measured fraction of edges kept, and the parameter a sparsifier is given does not determine it. Sampling schemes that clip per-edge probabilities at one retain fewer edges than their nominal parameter implies, and the shortfall grows with the skew of the score distribution. Local selection rules that give every vertex a quota cannot go below the retention that quota implies, whatever target they are given, which puts a hard floor under them on sparse graphs. Comparing two sparsifiers at equal nominal parameters therefore compares them at unequal amounts of deletion. We compare at matched realized retention and report the realized value everywhere.

A comparison must also be like for like in what is removed. A method that deletes vertices as well as edges scores its partition on a smaller and denser subpopulation, which is an easier problem, so the fraction of vertices retained belongs beside the fraction of edges.

G. Cost measured end to end

The sparsifier is a computation and belongs to the pipeline that uses it. The comparison is the whole pipeline, sparsifier included, against one run of the same detector on the original graph. Charging the sparsification to nobody can turn a loss into a saving on paper, and it was charged correctly in the original report [15]; Gottesbüren et al. [21] observe the same overhead in graph partitioning.

TABLE I
The protocol. Each control answers a specific way in which a comparison between a sparsified pipeline and an unsparsified baseline can favour the pipeline without any improvement having occurred. Section IV measures what each one is worth.

How the comparison can mislead	Control
Quality read off the sparsified graph, which admits higher modularity	Score both partitions on the original graph, Eq. (1)
Finer partitions score higher on agreement measures	Partition the original graph at a matched community count
Uncorrected measures pay for nothing, and pay more as parts multiply	Chance-corrected indices, and a size-matched random-partition floor
The pipeline has spent more compute than the baseline	Compute-matched restarts and the best of a fixed number
A degree-derived score acts through the degree sequence	Degree-preserving rewiring
A weighting is credited to similarity	Permute the weights across edges
Nominal and realized retention differ, and differ by method	Compare at measured retention; report floors and vertex coverage
Sparsification is charged to nobody	Time the whole pipeline, sparsifier included

TABLE II
The seven networks carrying the quality measurements, ordered by average degree. Sizes are of the largest connected component. GT marks the three with reference communities.

Network	n	m	d_{avg}	GT
com-Amazon	334,863	925,872	5.53	•
ca-HepTh	8,638	24,806	5.74	
com-DBLP	317,080	1,049,866	6.62	•
ca-CondMat	21,363	91,286	8.55	
email-Enron	33,696	180,811	10.73	
wiki-Vote	7,066	100,736	28.51	
email-Eu-core	986	16,064	32.58	•

IV. Experiments

Everything below runs under the protocol of Section III. The experiments answer three questions in order. What does the accounting cost, that is, how much of the reported benefit in this literature is a property of how it is measured. Where the pipeline is used in practice, does it help. And where it does help, what are the conditions.

A. Setup

a) Networks: The quality measurements use the seven real networks of Table II, chosen to span an order of magnitude in average degree while remaining tractable under a protocol that reruns every configuration with multiple seeds and computes several controls per cell. Three carry reference communities: email-Eu-core has department labels, and com-DBLP and com-Amazon carry the overlapping communities distributed with them. The mechanism measurements, which require only one partition per graph and no restarts, run on a larger suite of seventeen networks extending to 25 million edges, listed in Section ??. All graphs are reduced to their largest connected component and treated as simple and undirected.

Real networks cannot answer questions about where a transition lies, because their density cannot be varied while their community structure is held fixed. Two families of synthetic benchmark do that. LFR graphs [22] with 10^4 vertices and degree exponent 2.1 are generated at nominal average degrees of 10, 25, 50, 100 and 200, three graphs per configuration, matching the generator settings of the original synthetic sweep. Stochastic block models and degree-corrected block models are generated at the same degrees, with degree and mixing calibrated per cell to the realized values of the LFR graphs, in two variants each so that degree heterogeneity and community-size heterogeneity form a 2×2 factorial. Realized degree, realized mixing and degree coefficient of variation are recorded per graph and all results are reported against realized values.

b) Sparsifiers: Seven methods are tested, chosen so that a negative result cannot be attributed to the selection. Two are the methods whose quality claims motivate the pipeline: local sparsification, L-Spar [15], which lets each vertex keep the $\lceil d_v^e \rceil$ incident edges of highest neighbourhood Jaccard similarity, and the degree-based sampler DSpar [13]. Three more are the methods that the largest independent benchmark of sparsification ranks highest at preserving clustering structure, namely K-Neighbor, Local Degree and Local Similarity [19], [16], so that the arms include the strongest candidates chosen by someone other than us. L-Spar and Local Similarity are distinct methods despite the similar names, and L-Spar always refers to the method of Satuluri et al. The remaining two are a connectivity-preserving backbone, a spanning structure topped up to the retention target either at random or by Jaccard similarity, which cannot disconnect a graph by construction and is included as an instrument rather than as a competitor. Uniform random deletion supplies the no-information baseline wherever a selection rule has to be shown to be doing work.

Two classes are excluded and the exclusions bound what follows. Node-removing methods such as k-core peeling are not tested, because they score their partition on a smaller and denser vertex set, which is a different comparison requiring the coverage control of Section III-F. Learned sparsifiers [17] are not tested, because matching them on realized retention requires retraining at each operating point.

c) Detection algorithms: Four algorithms choose their own number of communities: Leiden [5] and Louvain [4], which maximize (3), Infomap [6], which minimizes a description length, and label propagation [?]. One takes the number as an input: Metis [?], through the pymetis bindings, which also imposes a balance constraint on part

TABLE III
Hard retention floors, as a fraction of edges. Below these values the method cannot run at all, whatever target it is given. Networks are ordered by average degree, which governs the floor. DSpar and uniform deletion have no floor.

Network	d_{avg}	Backbone	L-Spar	Loc. Deg.	K-Neigh.
com-Amazon	5.53	0.362	0.301	0.352	0.330
ca-HepTh	5.74	0.348	0.276	0.347	0.317
com-DBLP	6.62	0.302	0.244	0.301	0.277
ca-CondMat	8.55	0.234	0.186	0.234	0.218
email-Enron	10.73	0.186	0.159	0.186	0.179
wiki-Vote	28.51	0.070	0.067	0.070	0.069
email-Eu-core	32.58	0.061	0.053	0.061	0.060

TABLE IV
Uniform random edge deletion under the two accountings, Leiden, mean of three seeds. ΔQ_{self} is positive in every cell and $\Delta Q_{\text{transfer}}$ is negative in every cell. Uniform deletion uses no structural information, so the inflation comes from the objective rather than from any selection rule.

Network	Q_{base}	ρ	ΔQ_{self}	$\Delta Q_{\text{transfer}}$
ca-CondMat	0.729	0.800	+0.0111	−0.0164
ca-CondMat	0.729	0.900	+0.0053	−0.0064
email-Enron	0.611	0.800	+0.0052	−0.0162
email-Enron	0.611	0.900	+0.0011	−0.0086
com-DBLP	0.826	0.800	+0.0078	−0.0247
com-DBLP	0.826	0.900	+0.0040	−0.0098

sizes. Where Metis is used, k is fixed to the same value in the sparsified and unsparsified arms, so the two partitions have identical community counts by construction and no granularity effect is possible.

On real networks k has to be chosen, and we report both defensible choices: the community count a modularity optimizer returns on the original graph, and, on the three networks with reference communities, the number of those communities. The two differ by as much as a factor of sixteen and are reported separately throughout.

Of the algorithms behind the founding accuracy claim, Metis is tested here on every network. For Graclus and MLR-MCL we could find no maintained implementation that we were able to run, so MLR-MCL, the algorithm that claim is largest about, remains untested by us. This is the most consequential gap in the paper and Section V returns to it.

d) Operating points: Sparsifiers are compared at matched realized retention, with targets of 0.5 and 0.2. Four of the seven methods cannot reach the lower target on a sparse graph, because each vertex retains at least one incident edge, so a third operating point is added per network at the largest floor among the arms, ensuring one aggressive point at which all seven are comparable. Table III gives the floors, and they are a result in their own right rather than a nuisance parameter, since they bound how far the pipeline can be pushed on the graphs where it would need to pay off most.

e) Replication: Stochastic sparsifiers are run with independent sparsification seeds and stochastic detectors with independent detection seeds; deterministic methods are run once and reused. Baselines receive the two comparisons of Section III-D, the compute-matched restarts and the best of a fixed number. Where Metis is used the comparison is the worst sparsified run against the best baseline run over ten partitioner seeds, which is the strictest form available and the one used throughout for that detector.

f) Pre-registration: Each experiment has a design document written before its code, fixing what is measured, what is predicted, and what outcome would end that line of work. Several predictions were ours and failed, and where a result below contradicts a prediction we made, it is reported as such.

B. What the accounting costs

Each control in Section III exists because a comparison can favour the pipeline without any improvement having occurred. This subsection measures how much each one is worth. The measurements are of the evaluation and not of the sparsifiers, and they set the size of the correction that the rest of the paper applies.

a) Scoring on the sparsified graph: Table IV takes uniform random edge deletion, whose selection rule uses no information whatever, and scores it both ways on three networks at two retentions. Read off the sparsified graph it improves modularity in all six cells. The same partitions scored on the original graph are worse than the baseline in all six. The sign of the conclusion is opposite in every cell, and the gap widens as retention falls, which is what Section III-A predicts: the sparser the graph, the higher the modularity an optimizer can reach on it.

The effect is not particular to that sparsifier or that detector. Table V covers 63 configurations spanning three detection algorithms, seven networks and three sparsifier settings. Modularity read off the sparsified graph is positive in 60 of them and reaches +0.537; the same partitions scored on the original graph are positive in 15. Between the two accountings the sign of the conclusion differs in 45 of the 63 configurations. Eleven cells exceed a compute-matched baseline, all of them under label propagation, and they are not a subset of the 15: two sit below the plain baseline mean while beating a matched control that had drawn too few restarts, which is the failure the compute control addresses.

b) Granularity: Table VI gives the anatomy of a recovery gain on com-DBLP under Louvain. L-Spar at realized retention 0.503 raises average best-match F_1 from 0.102 to 0.193, an apparent gain of +0.091, while taking the number of communities of at least three members from 220 to 10,947. Partitioning the original graph to the same count reaches 0.300. The finer partition is worth more than twice what the sparsification appeared to be worth and requires no sparsifier. The same pattern holds for the degree-based arm on the same network, where the apparent change is negative and the matched control is positive. Leiden reproduces the sparsified figure to four decimal places and its own control, at $\gamma = 576$ and 10,812

TABLE V
The two accountings across 63 configurations, seven networks by three sparsifier settings under each algorithm. Both quantities are measured against the mean of the plain restarts, the weakest of the baselines in Section III-D; counts against the stricter best-of-N baseline appear in Section IV-C. A flip is a cell where $\Delta Q_{\text{self}} > 0$ and $\Delta Q_{\text{transfer}} < 0$, so that the two accountings disagree on the sign of the conclusion.

	Infomap	Louvain	Lab. prop.	Total
Cells	21	21	21	63
$\Delta Q_{\text{self}} > 0$	20	20	20	60
$\Delta Q_{\text{transfer}} > 0$	6	0	9	15
Flips	14	20	11	45
max ΔQ_{self}	+0.312	+0.260	+0.537	+0.537

TABLE VI
Granularity control on com-DBLP under Louvain, average best-match F_1 over reference communities of size at least three. $k_{\geq 3}$ counts communities of that size; γ is the resolution used on the original graph. Each sparsified arm is followed by its control, a partition of the original graph at the same $k_{\geq 3}$. The floor is a size-matched random partition at the arm's own $k_{\geq 3}$. Baseline and sparsified rows are means over three to six seeds; controls are single runs.

Condition	$k_{\geq 3}$	γ	F_1	floor
Original graph	220	1	0.102 ± 0.005	0.019
DSpar, $\rho = 0.500$	1,684	1	0.098 ± 0.002	0.049
matched control	1,657	24	0.125	
L-Spar, $\rho = 0.503$	10,947	1	0.193 ± 0.000	0.090
matched control	11,159	640	0.300	

communities, reaches 0.297, so the effect belongs to the modularity objective's default granularity and not to one optimizer's search.

Where the counts cannot be brought together, we draw no recovery conclusion. The comparability rule is $|\Delta k|/k < 25\%$, and the cells it excludes are reported as excluded rather than compared.

c) Chance: The floor column of Table VI carries the second artifact. A random partition with the same community size distribution scores 0.019 against the reference communities at 220 communities and 0.090 at 10,947, so a comparison that reads +0.091 from a fragmenting sparsifier is reading a floor that rose by 0.071 underneath it. The same measurement on com-Amazon gives 0.009 at 231 communities and 0.057 at 8,651, a 6.6× rise from fragmentation alone.

d) The compute budget: The registered compute-matched control buys less than it appears to. In a sweep of 48 cells over six networks, four retention targets and two samplers under Leiden, it bought exactly two restarts in every cell; across the 63 configurations of Table V it bought between 1 and 16, and 1 in 13 of the 18 cells on the two largest networks. Label propagation shows the consequence. Under the compute-matched control it is the strongest of the three algorithms in that table, with a mean change of +0.009; against the best of 20 plain restarts of the same baseline, 10 on the two largest networks, it is the weakest, at −0.106. On email-Enron its two apparent gains of +0.146 and +0.161 become −0.068 and −0.053. The compute-matched baseline drew four and five restarts in those two cells and still never sampled the algorithm's good mode, whose modularity is 0.523 against the matched control's 0.308.

e) Weighting credited to similarity: On email-Enron, weighting each edge by $1 + J(u,v)$, with J the Jaccard similarity of the endpoint neighbourhoods, raises modularity measured on the original graph to 0.6153 ± 0.0034 against a baseline of 0.6071 and a compute-matched best of 0.6051, a gain of +0.0102 at 3.7 times the pooled standard deviation. Permuting those same weights across edges, which preserves their distribution and destroys their placement, gives 0.6168 ± 0.0020, a gain of +0.0117 at 6.0 pooled standard deviations. The null reproduces the gain and exceeds it. On a degree-preserving rewiring of ca-CondMat, where the similarity signal is absent by construction, weighting still produces +0.0023 and clears the same significance test. What the weights contribute is heterogeneity in the optimizer's search and not information about communities.

f) Nominal retention: The parameter a sampler is given does not determine what it deletes. The scheme in common use clips per-edge probabilities at one, and across six networks at four targets between 12% and 36% of edges reach that clip, so the expected number retained falls below the nominal αm and saturates: on email-Eu-core the expected retention at $\alpha = 1.0$ is 0.665, and on wiki-Vote at $\alpha = 0.95$ it is 0.447. Three samplers that describe themselves by the same α realize different fractions. Sampling with replacement gives 0.33 to 0.52 across seventeen networks at a nominal 0.8, the clipped no-replacement scheme gives 0.37 to 0.68 across those six networks and four targets, and solving the scale factor by bisection on the same cells gives 0.70 to 0.95. A comparison at equal nominal parameters is a comparison at unequal deletion.

C. Where sparsification does not help

a) The objective: Table VII reports the result. Across 102 matched-retention cells, seven sparsifiers on seven networks under a modularity optimizer, not one cell improves modularity measured on the original graph, whether the baseline is the compute-matched control or the best of five plain restarts. The best cell in the entire grid loses 0.0105.

The sweep that isolates one method across its full retention range agrees. Local L-Spar on the same seven networks loses between 0.014 and 0.404 against a compute-matched baseline in every configuration that runs, with the loss growing as retention falls, and its three most aggressive configurations cannot be run at all because of the floors in Table III.

b) Under a fixed-k partitioner: The results above concern detectors that choose their own granularity. The condition under which sparsification does help, established in this paper on synthetic benchmarks, involves a detector

TABLE VII
Every matched-retention cell, Leiden, seven networks. ΔQ is measured on the original graph, Eq. (1), against the best of five plain restarts. Counts are cells with a positive value. The arms differ in how much they lose and not in whether they lose.

Arm	Cells	$\Delta Q > 0$	Best cell	Worst cell
K-Neighbor	18	0	−0.0105	−0.667
DSpar	18	0	−0.0116	−0.640
L-Spar	9	0	−0.0117	−0.407
Backbone, random fill	14	0	−0.0141	−0.214
Local Similarity	15	0	−0.0172	−0.311
Local Degree	14	0	−0.0288	−0.278
Backbone, Jaccard fill	14	0	−0.0290	−0.235
Total	102	0	−0.0105	−0.667

that takes the number of communities as an input, so the question is whether such a detector escapes the negative on these graphs.

It does not. Metis was run on all seven networks at the same operating points and under the same protocol, with ten partitioner seeds per cell and the worst sparsified run compared against the best baseline run. On the five networks with average degree between 5.5 and 10.7, no cell is positive: 90 cells, seven sparsifiers, two ways of choosing k, and the best of them loses 0.017. Fixing the community count removes the granularity artifact by construction and leaves the loss exactly where it was.

The two conventions for choosing k agree throughout, whether k is taken from the community count a modularity optimizer returns on the original graph or, on the three networks with reference communities, from the number of those communities. On com-Amazon the two values differ by a factor of sixteen, 306 against 5,000, and both give the same answer.

Above that range the picture changes, and Section IV-F takes it up: on the densest network we test, four of the seven sparsifiers produce a gain that survives the worst case over ten seeds.

c) Recovery: Agreement with reference communities behaves differently from the objective, and the difference is instructive rather than encouraging. Raw agreement often rises, and Table VI has already shown why: the sparsified partition is finer, the measure rewards that, and the floor beneath it rises at the same time. After both controls there is nothing left, on every labelled network but one.

d) Speed: The pipeline is not faster. Of the 63 configurations, four preserve quality in the sense of coming within 0.005 of the best-of-N baseline, and the fastest of those four runs at 0.66× the speed of clustering the original graph directly. Twelve configurations exceed 1.5×, and every one of them loses modularity, by 0.015 to 0.162. Timing only the detection step and giving the sparsifier away for free, the median speedup is 1.21×, 1.63× and 1.15× by algorithm, well short of the factor of two that halving the edge count suggests, because all three algorithms return more communities on the sparsified graph and do more work per pass.

D. Four ways the loss could be an artifact

A negative result is only as good as the explanations it excludes. Four are available, and each was tested rather than argued.

a) Fragmentation: Sparsification breaks graphs apart, and the optimizer may be penalized for being handed a shattered graph. The backbone arms test this directly, because they retain a spanning structure and cannot increase the number of connected components. They behave as designed: under the backbone with random fill no detected community is a singleton in any of its fourteen cells, and the fraction of vertices in communities of fewer than three members never exceeds 1.8%, where the shattering arms reach 86% on the same networks. The conclusion does not move. The backbone's best cell is −0.0141, worse than three of the arms that fragment freely. Removing half the edges costs modularity on the original graph whether or not the graph stays in one piece.

b) The choice of optimizer: Everything above uses a modularity optimizer, so the objective and the algorithm may be too closely coupled. Two sparsifiers across three further algorithms, one flow-based, one a local majority rule and a second modularity optimizer, give the 63 configurations of Table V. Against the best of N plain restarts, 60 are negative, with mean changes of −0.082 under Infomap, −0.046 under Louvain and −0.106 under label propagation. The three positive cells are label propagation on one network whose unsparsified baseline is pathological: its own best of twenty restarts reaches modularity 0.079 where a modularity optimizer on the same graph reaches 0.416. Relieving a known failure mode of one algorithm is not a benefit of sparsification.

c) A benign loss: The loss might be harmless if sparsification sheds peripheral vertices while leaving community cores intact. It does not. The fragments subdivide communities without mixing them, but they cut the most embedded vertices as readily as the least, and a resolution-matched partition of the original graph preserves cores better in sixteen of twenty comparisons. Restricting the damage to leaf and near-leaf vertices does not rescue it either.

d) The retention regime: The loss might be an artifact of pushing the methods too hard. It is not: it is already present at the mildest retention any of them can reach. On the three sparsest networks that is about a third of the edges, by Table III, and every arm loses there. The regime in which sparsification would pay for itself is not merely unprofitable on these graphs, it is unreachable.

One observation from the same arms does not belong to this list, because it supports no rival explanation. The backbone's two fills differ only in which edges top up the spanning structure, random or highest Jaccard similarity, and random fill scores better in ten of the fourteen paired cells. In this regime the similarity signal subtracts.

E. The exception

One result in this section survives every control we have.

On com-Amazon, L-Spar at realized retention 0.542 reaches average best-match F_1 of 0.402 against a baseline of 0.142. The granularity control does not remove it: partitioning the original graph to the same community count gives 0.319, and deliberately over-matching by 37 per cent more communities still gives only 0.372. The chance floor does not remove it either, and subtracting the floor widens the margin rather than narrowing it, because the floor rises with the community count. A second, independent instance appears in the same experiments from an unrelated method, Local Degree, on com-Amazon and com-DBLP.

What removes it is a different comparison. Both non-modularity algorithms reach higher agreement on com-Amazon without any sparsification at all, 0.465 under Infomap and 0.480 under label propagation, against the 0.402 that the sparsified modularity pipeline attains. The effect is real, and it is the modularity objective's default granularity being partially repaired by shattering the graph, in a metric that rewards fine partitions. It is not a capability that sparsification confers.

Two things about this cell are worth carrying forward. It coexists with a modularity loss of 0.079 in the same configuration, so the objective and the recovery measure move in opposite directions on the same partition of the same graph. And it is the reason the negative result of this section is stated as it is: no benefit on the objective anywhere, and no recovery benefit that survives comparison against a better algorithm run on the untouched graph. Section V takes up the dissociation as an open phenomenon.

F. The boundary

Everything so far is measured on one side of a line. The networks in Table II have average degrees between 5.5 and 32.6, and the detectors used for most of it choose their own granularity. Both conditions in this paper's claim are therefore held at one setting, and a negative result obtained that way says nothing about where the setting stops mattering. This subsection varies both.

a) What the setting requires: Three properties are needed. Density must vary over an order of magnitude while community structure is held fixed, which real networks cannot provide. Community membership must be known rather than inferred. And the granularity artifact of Section IV-B must be removed by construction rather than by control, which a detector taking k as an input does: both arms return the same number of communities by definition.

b) The transition: Table VIII reports, for LFR graphs at each average degree, the number of configurations in which L-Spar improves modularity measured on the original graph under Metis, and the number in which it improves chance-corrected agreement with the planted labels.

The two measures cross at different places. Recovery turns positive first, with half the configurations improving at average degree 24.6 and three quarters at 49.4, while the objective is still negative on average there and only becomes positive in the mean at 101.9. The location of the crossing was not chosen by us and was not known to us when the experiment was designed: it coincides with the degree at which the original authors reported their method beginning to overtake the unsparsified baseline.

TABLE VIII
Metis with k pinned to the planted community count, LFR, realized mixing near 0.6. Counts are configurations out of twelve in which L-Spar improves the quantity named, with the mean change beside them. Modularity is measured on the original graph. Because k is an input, both arms return identical community counts and no granularity effect is possible.

d_{avg}	$\Delta Q > 0$	mean ΔQ	$\Delta\text{AMI} > 0$	mean ΔAMI
11.2	0/12	−0.103	0/12	−0.216
24.6	0/12	−0.046	6/12	−0.072
49.4	8/12	−0.016	9/12	+0.042
101.9	10/12	+0.006	12/12	+0.116
220.9	12/12	+0.009	12/12	+0.093

The size of the effect should be read with the counts. At the degree where the objective crosses, the mean change is still negative and the positive cells are worth thousandths of a modularity point. The recovery gains are an order of magnitude larger, reaching a mean of +0.116.

c) Run-to-run variation: Metis is not deterministic across option seeds, so the decisive cells were repeated with ten seeds per graph and the worst sparsified run compared against the best unsparsified run. The comparison separates the two measures again. On recovery it is comfortable: at aggressive retention the worst case clears the baseline seed spread by factors of 8.0 to 14.4 at every degree at or above 49. On the objective it is not: the same cells clear it by factors between 1.0 and 5.8, and at the mildest retention several cells at 51, 201 and 261 are negative in the worst case. The recovery result survives a hostile reading of the seeds; the modularity result survives only at the aggressive operating points.

d) Generality across generators: LFR graphs are one family with one degree distribution, and the mechanism runs through a degree-derived or similarity-derived score, so the family could be doing the work. Repeating the arm on stochastic block models and on degree-corrected block models, calibrated cell by cell to the LFR graphs' realized degree and mixing, reproduces the sign flip on both. It also refutes the explanation we expected. We predicted that a degree-homogeneous generator would push the transition to higher density, since a degree-based selector has less to work with, and that the degree-corrected variant would sit closer to LFR. Neither holds. All four block-model variants cross at essentially the same place while their degree coefficient of variation spans an order of magnitude, from 0.14 to 1.47, and across sixty generator-by-degree cells the gain correlates with average degree at Spearman +0.65 and with degree variability at −0.26. Degree heterogeneity is not the operative variable.

e) Real networks: The same arm on the seven real networks places the transition without resolving what sits on the far side of it. Below average degree 11 there is nothing: 90 cells across five networks, seven sparsifiers, two ways of choosing k, and not one positive in the worst case over ten seeds. On wiki-Vote, at average degree 28.5, four of the seven sparsifiers produce a worst-case positive gain, the largest being +0.0124. On email-Eu-core, at 32.6, none does: two arms are positive in the mean and every worst case is negative. The transition on real graphs therefore lies between 10.7 and 28.5, and above it the effect is present on one of the two networks we can test.

f) What produces the gain: On the benchmark graphs the selection rule matters: degree-based and uniform sparsification at identical retention are negative at every degree except the two highest, where DSpar turns positive in 2 and 3 of twelve cells against L-Spar's 10 and 12. On wiki-Vote that separation disappears. The four arms that score individual edges locally all produce the gain, DSpar among them at +0.0076 against L-Spar's +0.0124, and the two connectivity-preserving backbones destroy it. What the gain requires is a rule that scores edges one at a time; which signal the rule uses matters less than we expected, and we do not identify the property that separates Local Degree, which is also local and is the worst arm on that network.

g) Why density: The denoising account makes a testable prediction. If sparsification helps by deleting edges that do not belong to the generative structure, its benefit should grow with the proportion of such edges. Injecting uniformly random edges into benchmark graphs while holding the planted labels fixed confirms it for similarity-based selection: recovery gain rises monotonically with the injected fraction in every one of the four configurations tested, and at average degree 50 with 20% retention it grows from +0.061 with no injected noise to +0.185 when the number of spurious edges equals the number of real ones. The gain does not vanish at zero noise, so denoising adds to an effect already present rather than being its sole cause, and a benchmark's noise is random by construction whereas a measurement process need not be. The experiment establishes the direction, not the magnitude in the field.

h) The same graphs, the other detector family: Running a free-granularity optimizer on the identical graphs gives a different answer, which is what makes the boundary a boundary rather than a threshold in density alone. Honest modularity gains appear in 28 of 144 configurations and none exceeds +0.005, within the spread of five random restarts. Recovery gains do appear and grow with degree, but they do not survive the granularity control: partitioning the original graph at a resolution tuned to the sparsified partition's community count recovers the planted communities as well or better in 104 of the 144 configurations where that comparison is defined. What looked like a benefit of sparsification is the benefit of a finer partition, available without touching the graph.

i) Three things the positive result does not say: Sparsification repairs a weaker detector rather than producing the best available partition. At average degree 49, Metis with L-Spar reaches AMI 0.75 against its own baseline of 0.62, a real improvement to that algorithm, while a resolution-tuned optimizer on the untouched graph reaches 0.94 on the same graphs. The defensible statement is that similarity-based selection repairs a fixed-k cut partitioner on dense graphs, not that it improves community detection.

The quality gain is not a speed gain in our implementation. Computing exact pairwise similarity costs more than it saves, and the deficit grows with density because the similarity computation grows faster than the detection it accelerates: the median end-to-end speedup falls from 0.75× at average degree 10 to 0.26× at 200. The original speedups were obtained with an approximate similarity computation reported as two orders of magnitude cheaper, measured against clustering runs lasting hours, so both cost bases differ from ours by enough to explain the discrepancy without disagreement about what should be counted.

One part of the original report does not reproduce. At high mixing, where the planted structure is barely detectable and baseline recovery sits between 0.06 and 0.14, no gain appears at any degree: across 104 configurations L-Spar improves modularity in 9, and the mean change is negative at every degree. The original sweep reports its largest improvement in that regime. Whether this is a genuine non-reproduction or a floor effect of the metrics we use cannot be settled at that mixing level, and we report it unresolved.

V. Discussion

VI. Conclusion

References

[1] S. Fortunato, "Community detection in graphs," Physics Reports, vol. 486, no. 3–5, pp. 75–174, 2010.

[2] S. Fortunato and M. E. Newman, "20 years of network community detection," Nature Physics, vol. 18, no. 8, pp. 848–850, 2022.

[3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical Review E, vol. 69, no. 2, p. 026113, 2004.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008.

[5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, pp. 1–12, 2019.

[6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences, vol. 105, no. 4, pp. 1118–1123, 2008.

[7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," Internet Mathematics, vol. 6, no. 1, pp. 29–123, 2009.

[8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, 2008, pp. 563–568.

[9] A. A. Benczúr and D. R. Karger, “Randomized approximation schemes for cuts and flows in capacitated graphs,” SIAM Journal on Computing, vol. 44, no. 2, pp. 290–319, 2015.
[10] D. A. Spielman and S.-H. Teng, “Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems,” SIAM Journal on Matrix Analysis and Applications, vol. 35, no. 3, pp. 835–885, 2014.
[11] U. von Luxburg, “A tutorial on spectral clustering,” Statistics and Computing, vol. 17, no. 4, pp. 395–416, 2007.
[12] J. Shi and J. Malik, “Normalized cuts and image segmentation,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, pp. 888–905, 2000.
[13] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, “Dspar: An embarrassingly simple strategy for efficient GNN training and inference via degree-based sparsification,” Transactions on Machine Learning Research, 2023.
[14] L. Lovász, “Random walks on graphs,” Combinatorics, Paul Erdős Is Eighty, vol. 2, no. 1–46, p. 4, 1993.
[15] V. Satuluri, S. Parthasarathy, and Y. Ruan, “Local graph sparsification for scalable clustering,” in Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 721–732.
[16] M. Hamann, G. Lindner, H. Meyerhenke, C. L. Staudt, and D. Wagner, “Structure-preserving sparsification methods for social networks,” Social Network Analysis and Mining, vol. 6, no. 1, p. 22, 2016.
[17] H.-Y. Wu and Y.-L. Chen, “Graph sparsification with generative adversarial network,” in 2020 IEEE International Conference on Data Mining (ICDM), 2020.
[18] V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, and J. Lin, “SimClusters: Community-based representations for heterogeneous recommendations at Twitter,” in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2020, pp. 3183–3193.
[19] Y. Chen, H. Ye, S. Vedula, A. Bronstein, R. Dreslinski, T. Mudge, and N. Talati, “Demystifying graph sparsification algorithms in graph properties preservation,” Proceedings of the VLDB Endowment, vol. 17, no. 3, pp. 427–440, 2024.
[20] M. Molloy and B. Reed, “A critical point for random graphs with a given degree sequence,” Random Structures & Algorithms, vol. 6, no. 2–3, pp. 161–180, 1995.
[21] L. Gottesbüren, N. Maas, D. Rosch, P. Sanders, and D. Seemaier, “Linear-time multilevel graph partitioning via edge sparsification,” in 33rd Annual European Symposium on Algorithms (ESA), 2025, arXiv:2504.17615.
[22] A. Lancichinetti, S. Fortunato, and F. Radicchi, “Benchmark graphs for testing community detection algorithms,” Physical Review E, vol. 78, no. 4, p. 046110, 2008.
[23] R. Guimerà, M. Sales-Pardo, and L. A. N. Amaral, “Modularity from fluctuations in random graphs and complex networks,” Physical Review E, vol. 70, no. 2, p. 025101, 2004.
[24] N. X. Vinh, J. Epps, and J. Bailey, “Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance,” Journal of Machine Learning Research, vol. 11, pp. 2837–2854, 2010.